
A Deterministic Numerical Stack

N. E. Davis `~lagrev-nocfep`
ZeroAttest, Groundwire Foundation

Abstract

Urbit’s deterministic functional architecture requires that all computations be reproducible to the bit across hosts. This article describes the design and implementation of a deterministic numerical stack, including a software-defined BLAS library, a software-defined universal number (unum) library, a rank- N array library with type dispatch, and a transcendental functions library. We benchmark Chebyshev basis functions with Cody–Waite range reduction, Taylor series expansions, and other numerical methods.

Contents

1	Introduction	2
2	Background and Literature	3
2.1	BLAS as a pseudo-standard	4
2.2	Sources of floating-point nondeterminism . . .	4
2.3	Reproducibility	5
3	Deterministic Architecture	6
3.1	The numerical stack	7
3.2	SoftBLAS and SoftUnum	9
3.2.1	SoftBLAS	9
3.2.2	SoftUnum	12

26	4 Types	12
27	5 Contributions	13
28	6 Benchmarks	15
29	6.1 Verification	15
30	6.2 The price of determinism	22
31	7 Future Work	23
32	8 Conclusion	26

33 1 Introduction

34 Urbit is a deterministic functional operating system whose state
 35 is a pure function of its event log (~sorreg-namtyv et al., 2016).
 36 An event log replay must recover the state exactly, on any
 37 conforming runtime, indefinitely. This is not a performance
 38 nicety; it is the property that makes the system auditable, up-
 39 gradeable, and peer-verifiable. Determinism is a load-bearing
 40 property all the way down.

41 Numerical computing is one of the most slippery places for
 42 determinism to leak through its abstractions. Standard IEEE 754
 43 floating-point arithmetic is not associative, as an example, but
 44 the situation is much more dire. The IEEE 754 standard (, 2019)
 45 specifies the result of each individual operation under a given
 46 rounding mode, but a real numerical program is a composition
 47 of operations, and the standard says little about composition.
 48 A fused multiply-add computes $a \cdot b + c$ with a single rounding
 49 where two separate operations would round twice; a vector-
 50 ized dot product sums partial results in an order chosen by the
 51 hardware; and a threaded BLAS reassociates a reduction differ-
 52 ently depending on how many cores happened to be available.¹
 53 Each of these is standards-conformant while none is determin-
 54 istic across hosts.

¹Urbit computing is single-threaded which removes one possible source of nondeterminism.

The treatment of floating-point representation in Hoon by ~lagrev-nocfep (2024) established the foundation for Urbit’s deterministic numerical computing stack. This article expounds both the state of the art of Urbit’s basic numerical packaging and the next layer: dense linear algebra, where the nondeterminism of conventional BLAS and IEEE 754 floating-point mathematics are acute and consequential. We introduce SoftBLAS,² a software-defined BLAS package; SoftUnum,³ a software-defined universal number (unum) package; and other components of a C-based numerical stack that is deterministic and compliant with the Hoon specification. We further describe the “ray” data structure which facilitates the type dispatch for different numeric representations while preserving identity between Nock-specified atoms and C-recognized arrays.

The central conceit is architectural. In conventional high-performance computing, C (or Fortran) takes primacy: the C implementation is what runs, and correctness is in some sense whatever the C does.⁴ Urbit inverts this relationship. The Hoon/Nock ISA implementation is the specification; the C jet is an optimization that is *obligated* to produce the specification’s exact output for every input, and should be discarded silently whenever it cannot. Determinism then is not merely a testable, desirable property, but a mandate the Nock-side architecture cannot express the violation of.

2 Background and Literature

Although the solution of partial differential equations is one of the oldest significant applications of computing,⁵ no standard library emerged for decades—unsurprising given the churn of hardware architectures, the diversity of numerical methods, the lack of a common interface definition, the rapid evolution of programming languages, and the active development of lin-

²urbit/SoftBLAS (Urbit Foundation, 2024).

³sigilante/SoftUnum (~lagrev-nocfep, 2026).

⁴Verification and validation apply, of course.

⁵Charney, Fjortoft, and Neumann (1950).

ear algebraic algorithms.⁶ Given that situation, rigorous bit-wise determinism was a concern of vanishingly small importance, whether cross-platform or even for serial runs on the same machine. However, over time the advent of high-performance computing, the emergence of BLAS as a pseudo-standard, and the increasing importance of reproducibility in numerical computing have made determinism both more pressing and more achievable.

2.1 blas as a pseudo-standard

The Basic Linear Algebra Subprograms originate with Lawson, Hanson, Kincaid, and Krogh's 1979 specification of vector operations on strided arrays (Lawson et al., 1979), later extended to matrix-vector (Level 2) and matrix-matrix (Level 3) routines. The levels track both chronology and (naïve) asymptotic cost: Level 1 is $O(n)$, Level 2 is $O(n^2)$, Level 3 is $O(n^3)$. BLAS is less a standard than a pseudo-standard—a widely honored set of names and calling conventions (`?axpy`, `?gemv`, `?gemm`, with type prefixes `s`, `d`, `c`, `z`) with many independent implementations. (The original BLAS predated floating-point standardization as IEEE 754 by several years.) That plurality is exactly the problem for a consensus system: two conforming BLAS libraries routinely disagree in the low bits, and the same library disagrees with itself across machines and thread counts. Despite this fuzziness, BLAS quickly took its place as the bedrock layer of numerical and scientific computation.

2.2 Sources of floating-point nondeterminism

Cross-host divergence in a BLAS computation has a small number of recurring causes:

- *Reduction order.* Floating-point addition is not associative, so a dot product or a `gemm` inner loop summed in a different order yields a different rounded result; tiled and multithreaded implementations choose that order at run time.

⁶Even the size of a byte took decades to standardize.

- 119 • *Fused multiply-add.* Hardware that contracts $a \cdot b + c$
120 to one rounding produces different bits than two round-
121 ings, and whether contraction happens is a compiler-
122 and target-dependent decision.
 - 123 • *Vectorization and instruction selection.* SIMD horizontal
124 sums, x87 80-bit intermediates, and differing libm-style
125 kernels for fast paths all perturb the last bits.
 - 126 • *Transcendental functions.* The elementary functions are
127 not specified to the bit by IEEE 754 at all, and every plat-
128 form libm differs in its implementation.
- 129 ~lagrev-nocfep (2024) further enumerated other reasons that
130 contemporary floating-point mathematics cannot be consid-
131 ered deterministic even when single-threaded on a particular
132 hardware target and instruction set architecture.

133 2.3 Reproducibility

134 Numerical determinism, or for our purposes synonymously re-
135 producibility, requires that a computation yield the same result
136 to the bit regardless of the platform on which it is carried out.
137 This would not necessarily require the same algorithmic im-
138 plementation in all cases (since all correct algorithms should
139 converge on the true value given enough runway), but in prac-
140 tice floating-point quirks force this consideration in almost all
141 cases.

142 Projects like ReproBLAS have yielded reproducible sum-
143 mation (independent of summation order) through error-free
144 transformations and pre-rounding (Demmel and Nguyen, 2013).
145 Intel’s Conditional Numerical Reproducibility offers run-to-run
146 consistency within a fixed code path under documented con-
147 straints (Intel Corporation, 2023). The PLASMA and MAGMA pro-
148 jects (Agullo et al., 2009) took the opposite design stance for
149 performance: dynamic, dependency-driven scheduling that de-
150 liberately does not fix reduction order, which is excellent for
151 throughput and fatal for bit-reproducibility. On the scalar side,
152 correctly rounded elementary-function libraries—CR-LIBM
153 (Daramy et al., 2003), RLIBM (Lim and Nagarakatte, 2021), and

the CORE-MATH project (CORE-MATH Project, 2022)—show that the transcendentals can be pinned down to the last bit, at a cost.

Urbit is distinguished by its requirement that determinism be not one option among many but a first-class feature of the OS and runtime. This requirement is structurally enforced by the Nock ISA-jetted runtime relationship. Jets utilize relatively faster code paths and reductions but produce the identical result as the slow path: the specification is authoritative. Thus linear algebra techniques, even those following the BLAS pseudo-standard, must follow a reproducible fast path on all supported architectures.

3 Deterministic Architecture

A Nock computation is a pure function from noun to noun. A *jet* is a native routine registered against a named Hoon core; when the interpreter evaluates that core, it may run the jet instead of the Nock. The iron discipline that makes this sound is simple: the jet must produce the same noun as the Hoon, for every input, or it is a bug in the runtime. A divergence between jet and specification is termed a *jet mismatch* and is treated as a correctness failure, not a numerical tolerance.

The Hoon (or its compiled result as Nock) is therefore the specification, in the strong sense that it defines what the right answer *is*. The C jet is an accelerant with no semantic authority. In some cases, the jet may decline: if a native routine returns the sentinel `u3_none`, the interpreter falls through and evaluates the Hoon directly. The consequence for numerics is that an arithmetic kind with no jet, or with a jet that punts on some inputs, is not unsupported—it is merely slow. The specification still runs, and the answer is still defined and still deterministic, because Hoon over exact nouns is deterministic by construction.

This crystallizes, or perhaps even inverts, the usual relationship between correctness and performance. In conventional HPC, one has an optimized C or Fortran program, the optimized C arguably *is* the program and its source reference implementation is documentation (to say nothing of its docu-

mentation). There may of course be bugs in the compiled code, but the optimized program is the concrete artifact. By contrast in Urbit, the optimized C program must justify itself against an independent authoritative definition. Determinism follows from two facts: the specification is a pure function of its inputs, and the jet is contractually bit-identical to the specification. A host without the jet computes the same bits more slowly; a divergent jet is non-conforming and must be repaired.

For our linked implementations like SoftBLAS, this means the library is never permitted to be “approximately” the specification. A `gemm` jet that summed its inner products in a hardware-chosen order would be a mismatch even if every individual rounding were IEEE 754-correct, because the Hoon specification sums them in one fixed order and the jet must reproduce that number.⁷ Our BLAS must satisfy the constraint of determinism to be admissible as a jet at all; thus, SoftBLAS was developed on top of SoftFloat (Hauser, 2018). This yields correctness and consistency rather than hardware-optimized performance.

3.1 The numerical stack

Urbit natively supports IEEE 754 floating-point mathematics in its `%base` distribution, and through the supplied numerical stack additionally supports unums (posits and quires); complex IEEE 754; two’s complement integers; and fixed-point mathematics.

The numerical libraries are layered as follows; each layer is Hoon-normative with optional, independently verifiable C jets.

1. *Libraries*. Imported from `/lib` at compile time.

- (a) `/lib/math`. The basic IEEE 754 arithmetic and transcendental functions library supports four precisions (16-bit `@rh`, 32-bit `@rs`, 64-bit `@rd`, and 128-bit `@rq`) across a battery of functions like exponentiation and logarithm, trigonometric and inverse-

⁷Summation will necessarily exhibit order-dependent drift at the bit level.

- 224 trigonometric functions, and rounding and com-
 225 parison utilities. Transcendental functions are
 226 implemented as Chebyshev minimax polynomi-
 227 als with Cody–Waite range reduction (Cody and
 228 Waite, 1980). Jets run on SoftFloat (Hauser,
 229 2018). Originally composed by `~lagrev-nocfep`
 230 and `~mopfel-winrux`.
- 231 (b) `/lib/twoc`. A two’s-complement integer arith-
 232 metic package supporting scalar operations. Jet-
 233 ted with an internal library against GMP. Originally
 234 composed by `~litlep-nibbyt`.
- 235 (c) `/lib/fixed`. Fixed-point (Q-format) arithmetic at
 236 power-of-two total widths. Unjetted beyond stan-
 237 dard integer arithmetic jets because de facto in-
 238 teger mathematics hold. Originally composed by
 239 `~lagrev-nocfep`.
- 240 (d) `/lib/unum`. 2022-standard posit arithmetic (Posit
 241 Working Group, 2022) at five widths (8-bit `@rpb`; 16-
 242 bit `@rph`; 32-bit `@rps`; 64-bit `@rpd`; 128-bit `@rpq`, i.e.
 243 `posit8` through `posit128`), `es = 2` throughout, with
 244 quire accumulation. See Section 6.1 on oracle cov-
 245 erage. Jetted with SoftUnum. Originally composed
 246 by `~lagrev-nocfep`.
- 247 (e) `/lib/complex`. Complex arithmetic over the four
 248 IEEE 754 precisions, riding on the scalar float layer.
 249 Originally composed by `~lagrev-nocfep`.
- 250 2. *Lagoon (Linear ALgebra in hOON)*. A rank- N array li-
 251 brary (a `$ray`) carrying a `$kind` field that dispatches to
 252 the appropriate arithmetic: `%i754`, `%uint`, `%int2`, `%unum`,
 253 `%cplx`, `%fixp`. Storage is row-major and identical in mem-
 254 ory layout to underlying runtime arrays; the `%i754` Level-
 255 3 path is jetted through SoftBLAS and the `%unum` path is
 256 jetted through SoftUnum. Composed by `~lagrev-nocfep`
 257 and `~mopfel-winrux`.
- 258 3. *Saloon (Scientific ALgorithms in hOON)*. Scientific algo-
 259 rithms over Lagoon rays. Types correspond to the La-
 260 goon field. Element-wise transcendentals are dispatched

through `/lib/math`, and dense routines such as the Jacobi symmetric eigendecomposition.⁸ Composed by `~lagrev-nocfep` and `~mopfel-winrux`.

4. *Maroon (MAchine leaRning in hOON)*. An inchoate machine learning library built atop Lagoon and Saloon.

The single most consequential design choice is Lagoon’s `$kind` dispatch with fall-through (Listing 1). One basic array type serves six arithmetics; adding a seventh requires only a Hoon implementation, and a jet for it is optional and separately auditable. This is architecturally distinct from NumPy (which supplies one `dtype` per array with no in-band kind tag); from Julia (which has compile-time specialization per concrete type); and from the APL/J/K tradition (which supply a uniform numeric tower with no explicit kind dispatch). Because an unjetted kind still runs its Hoon, the specification is authoritative for every kind, jetted or not. Two other features of `$ray`’s design call attention: the `MSB` pinned 1 bit, which preserves otherwise-stripped leading zeroes in atoms and thus makes the atom representation exactly match the expected C memory layout of an array; and the `tail=*` arbitrary noun specialization field, which preserves jet axis addresses in case of future extension.⁹

3.2 SoftBLAS and SoftUnum

At the current time, two external numerical libraries facilitate the runtime jets: SoftBLAS and SoftUnum. Both are software-defined, deterministic, and cross-platform.¹⁰

3.2.1 SoftBLAS

SoftBLAS is built on Berkeley SoftFloat 3e (Hauser, 2018). Following SoftFloat, it requires a C99-compliant host and passes

⁸Broadly speaking, `Lagoon::NumPy::Saloon::SciPy`.

⁹For `%cplx`, `bloq` is the $\log_2$ of the packed-pair element width (both components combined). Thus `@cs` (two `@rs` components) has `bloq=6`, while each component `@rs` has `bloq=5`.

¹⁰Urbit mathematics are supported internally by the GNU Multiple Precision Arithmetic Library and SoftFloat.

Listing 1: Lagoon base types. From /sur/lagoon.

```

+$ ray                :: $ray:  n-dimensional array
$:  =meta             ::  descriptor
    data=@ux          ::  data, row-major order
==

5  ::
+$ prec  [a=@ b=@]   ::  fixed-point precision
+$ meta                ::  $meta:  metadata for a $ray
$:  shape=(list @)    ::  list of dimension lengths
    =bloq              ::  logarithm of bitwidth
10   =kind             ::  name of data type
    tail=*             ::  per-kind specialization data
==

::
+$ kind                ::  $kind:  type of array
    scalars
15   $? %i754          ::  IEEE 754 float
    %uint              ::  unsigned integer
    %int2              ::  2s-complement integer
    (/lib/twoc)
    %unum              ::  unum/posit (/lib/unum)
    %cplx              ::  complex (/lib/complex)
20   %fixp             ::  fixed-point Q a.b
    (/lib/fixed)
==

::
+$ baum                ::  $baum:  ndarray with metadata
$:  =meta             ::
25   data=ndray        ::
==

::
+$ ndarray             ::  $ndray:  n-dim array as list
    $@ @              ::  single item
30   (list ndarray)    ::  nonempty list of children

```

all floating-point quantities by value. Every arithmetic primitive used by every routine is a SoftFloat call: there is no path to a hardware floating-point instruction, and therefore no path to hardware-dependent rounding, contraction, or intermediate precision. For reasons enumerated by `~lagrev-nocfep` (2024), this is one of the only current methods which guarantees that the output of a floating-point computation is a pure function of its inputs, and therefore deterministic across hosts.¹¹

SoftBLAS covers the four IEEE 754 binary precisions used across the stack: half (h), single (s), double (d), and quadruple (q).¹² Routines follow the BLAS prefixing convention extended to the half and quad types. Real-valued routines across all three levels supply a subset sufficient for Lagoon requirements as of the cited release; complex-valued routines are in progress but sufficient to support Lagoon’s requirements.

Table 1: SoftBLAS coverage by level (real-valued).

Level	Cost	Representative routines
1	$O(n)$	?dot, ?axpy, ?scal, ?nrm2, ?asum, etc.
2	$O(n^2)$	?gemv
3	$O(n^3)$	?gemm

The rounding mode is a global, `softblas_roundingMode` (a `char` typedef declared in `softblas.h`) interpreted by the underlying SoftFloat layer.¹³ The admissible values are the SoftFloat 3e rounding modes (round to nearest even, round toward zero, round toward $+\infty$, round toward $-\infty$, and round to nearest-away).¹⁴ The default is round to nearest-even, %n.

¹¹ReproBLAS in contrast uses a hardware floating-point path by binning and accumulation, making reduction order irrelevant but still subject to hardware-dependent rounding (Demmel and Nguyen, 2013). SoftBLAS is slower but deterministic across platforms; it makes the fewest assumptions about the host.

¹²Complex routines are supplied at half (i), single (c), double (z), and quadruple (v) precisions.

¹³The transcendental function implementations ignore the rounding mode, which thus applies only to the arithmetic operations.

¹⁴IEEE 754-2008 introduced another mode, round to nearest ties to max magnitude, which is not included in Urbit’s toolset.

311 The Hoon specification names the rounding mode in the `%base`
 312 distribution, and the jet is required to honor it.

313 SoftBLAS satisfies three conditions that conventional BLAS
 314 libraries do not:

315 1. *Fixed reduction order.* Every reduction (dot products, `gemm`
 316 inner loops, norms) sums in one canonical order, identical
 317 to the order the Hoon specification uses. No tiling or
 318 threading is permitted to reassociate it. (Urbit runtimes
 319 are, in any case, single-threaded at the time of writing.)

320 2. *No contraction.* Multiply-add is two roundings, never a
 321 fused one, because the specification rounds twice. There
 322 is no FMA path, which as `~lagrev-nocfep` (2024) exhib-
 323 ited is prone to nondeterminism.

324 3. *Software rounding.* All rounding is performed by Soft-
 325 Float in software, so it is identical on every host regard-
 326 less of native floating-point behavior.

327 3.2.2 SoftUnum

328 SoftUnum is a software-defined universal number (unum) li-
 329 brary that implements the 2022 posit standard (Posit Working
 330 Group, 2022). It supports five widths (8, 16, 32, 64, and 128
 331 bits) with `es = 2` throughout. The quire accumulator is imple-
 332 mented in software and is bit-identical across hosts. The library
 333 is jetted into Lagoon’s `%unum` kind.

334 4 Types

335 The mathematical base types are enumerated in Listing 2. These
 336 have occasioned the introduction of several new atom auras,
 337 given in Table 2. We have not introduced an aura for two’s-
 338 complement integers at the current time, but these will likely
 339 nest under `@s` (signed integer) in the future. The `@rqb` through
 340 `@rqq` auras are quire accumulators for the corresponding posit

Listing 2: Mathematical base types. From `/sur/lagoon`.

```

+$ kind          :: $kind: type of array
    scalars
$? %i754         :: IEEE 754 float
    %uint        :: unsigned integer
    %int2        :: 2s-complement integer
    (/lib/twoc)
5    %unum        :: unum/posit (/lib/unum)
    %cplx        :: complex (/lib/complex)
    %fixp        :: fixed-point  $\mathbb{Q}$  a.b
    (/lib/fixed)
==

```

widths, and are intended for internal algorithms not general use.¹⁵

Aura types have been named after the target width in the cases of complex numbers and quires.

5 Contributions

This deterministic numerical stack contributes several properties to the broader discussion of numerical determinism and reproducibility. Determinism is a structural property of Urbit: while reproducible floating point is a standing goal of several parties (Demmel and Nguyen (2013); Intel Corporation (2023)), Urbit contributes a first-class philosophical requirement.¹⁶

Although not novel, the library covers a broad range of bit widths and precisions, including half-precision and quadruple-precision, which are not widely supported in other numerical computing libraries. Likewise we provided faithfully-rounded elementary functions at most precisions and quantify their accuracy.

¹⁵A quire is a $16n$ -bit two's-complement fixed-point accumulator with scale $2^{-q_{\text{scale}}}$ ($q_{\text{scale}} = 8n - 16$): a posit value v contributes the exact integer $v \times 2^{q_{\text{scale}}}$.

¹⁶Jet co-design methodology is treated separately; see `~lagrev-nocfep`, pp. TODO-TODO in this volume, for a companion article.

Table 2: Existing and new atom auras. From `/sur/lagoon`.

Atom aura	Description
<code>@rh</code>	16-bit IEEE 754 float
<code>@rs</code>	32-bit IEEE 754 float
<code>@rd</code>	64-bit IEEE 754 float
<code>@rq</code>	128-bit IEEE 754 float
<code>@rpb</code>	8-bit posit
<code>@rph</code>	16-bit posit
<code>@rps</code>	32-bit posit
<code>@rpd</code>	64-bit posit
<code>@rpq</code>	128-bit posit
<code>@rqbH</code>	8-bit quire (128-bit accumulator for <code>@rpb</code>)
<code>@rqhI</code>	16-bit quire (256-bit accumulator for <code>@rph</code>)
<code>@rqsJ</code>	32-bit quire (512-bit accumulator for <code>@rps</code>)
<code>@rqdK</code>	64-bit quire (1024-bit accumulator for <code>@rpd</code>)
<code>@rqqL</code>	128-bit quire (2048-bit accumulator for <code>@rpq</code>)
<code>@ch</code>	32-bit complex (two 16-bit floats)
<code>@cs</code>	64-bit complex (two 32-bit floats)
<code>@cd</code>	128-bit complex (two 64-bit floats)
<code>@cq</code>	256-bit complex (two 128-bit floats)

358 Lagoon’s heterogeneous-kind array design with a single
 359 `$ray` type is a novel approach to array libraries, allowing for
 360 in-band kind dispatch and fall-through to specification. As
 361 mentioned previously, this permits more flexibility and exten-
 362 sibility in the base representation than NumPy, Julia, or the
 363 APL/J/K tradition. The `kind` field allows for easy addition of
 364 new numeric types without requiring changes to the underly-
 365 ing array structure, and the fall-through mechanism ensures
 366 that un-jetted kinds can still be computed correctly using the
 367 Hoon specification.

6 Benchmarks

While a software-defined floating-point library will never match the performance of a hardware-accelerated one, we have nevertheless seen impressive speedups by refining the algorithms and the runtime jets. The benchmarks reported here are for a single-threaded, single-core execution on a modern 64-bit compiler CPU (AArch64, Apple M4 Max). They should serve as order-of-magnitude indications on contemporary hardware.

The first benchmark results tabulate what the jetted system buys over interpreting the Hoon specification directly. Table 3 reports jetted versus interpreted time for IEEE 754 `/lib/math`. For most trials, 100,000 runs were evaluated and averaged.¹⁷ For the very slowest functions (e.g. half-precision Taylor series inverse sine), this number was modulated downwards substantially.¹⁸ The original `/lib/math` and Saloon implementations of transcendentals were naïve Taylor series expansions and serve as a worst-case demonstration. Accuracies are comparable for the number of Taylor series terms utilized to achieve an appropriate relative tolerance, but the Chebyshev basis functions are uniformly faster and more correct. All values reported are measured within a running Urbit ship (rather than from an independent test harness).

The `/lib/unum` transcendentals are currently implemented as Chebyshev basis expansions like the `/lib/math` operations. Results are included in Table 4.

6.1 Verification

The determinism architecture reduces correctness to one question per routine: does the jet equal the specification on every input? We answer it insofar as possible with oracles external to both the Hoon and C implementations. SoftFloat by and large served as the oracle for the IEEE 754 floating-point operations

¹⁷Urbit memoizes aggressively, so countermeasures were taken to prevent caching effects. Inputs were varied every iteration and the result of each call was folded into a returned accumulator in order that neither input nor whole loop is a cacheable constant noun.

¹⁸Exhaustive results are available at `urbit/numerics@main:benchmark`.

(real and complex), while the posit mathematics were verified against two independent references: SoftPosit (Leong, 2018) (posit8, posit16, posit32) and the Universal library (Omtzigt et al., 2020) (posit64, posit128).

The quire’s defining property—exact accumulation until a single final rounding—is checked directly against an exact rational sum.

Elementary functions are checked against MPFR (Fousse et al., 2007) at high precision. At half precision the input space is 2^{16} values and the test is exhaustive; at single precision the 2^{32} space is likewise tested in full. At double and quadruple precision, where exhaustive testing is infeasible, we report maximum observed unit in the last place (ULP) error over a sampled and edge-weighted input set rather than a guarantee. We claim a faithful-rounding (≤ 1 ULP) result where the testing supports it and a measured bound where it does not.

We report the errors of the transcendental functions in terms of ULP error, which is a standard measure of floating-point accuracy.¹⁹ The maximum observed ULP error for each function is reported against an independent oracle, which is typically a high-precision library such as `mpmath`. The results are summarized in Tables 5 and 6.

The libraries instrumented with the Chebyshev basis are designed to produce “faithfully rounded functions” rather than “correctly rounded functions” for composed operations, while dedicated (non-composed) kernels frequently do better still. The distinction is that a faithfully rounded result is within 1 ULP of the exact result (Rump, Ogita, and Oishi, 2008), while a correctly rounded result is the exact representable value closest to the exact result. Faithful rounding is often sufficient for numerical applications and is generally faster to compute than correct rounding. Truly correct rounding is often prohibitively expensive, especially for transcendental functions, and can re-

¹⁹ULP is defined as the distance between two adjacent representable floating-point numbers, although the exact definition can vary slightly depending on the author; we follow Muller et al. (2018). A very good ULP error is typically 0.5 or less, which results from a correctly rounded library instead of a faithfully rounded one. Production `libm` often offers merely 1–4 maximum ULP (e.g. GNU `libc` claims “within a few ULP”, Free Software Foundation (2024)).

Table 3: Elapsed times and relative slowdowns of IEEE 754 jetted Chebyshev functions for double-precision @rd atom operations with `/lib/math`, interpreted Chebyshev functions, and interpreted Taylor series functions. Slowdown is reported relative to the fastest case, the jetted Chebyshev functions.

Arm	Chebyshev (jet)	Chebyshev (int.)	Taylor (int.)	C. slowdown	T. slowdown
<code>+exp</code>	1.67 μ s	296 μ s	11.0 ms	177 ^x	6,565 ^x
<code>+log</code>	1.76 μ s	164 μ s	13.3 ms	94 ^x	7,600 ^x
<code>+sin</code>	1.95 μ s	278 μ s	0.94 ms	143 ^x	480 ^x
<code>+cos</code>	1.97 μ s	275 μ s	0.94 ms	140 ^x	476 ^x
<code>+tan</code>	4.53 μ s	241 μ s	1.30 ms	53 ^x	288 ^x
<code>+atan</code>	2.23 μ s	47.6 μ s	109 ms	21 ^x	48,784 ^x
<code>+atan2</code>	2.72 μ s	56.2 μ s	11.5 ms	21 ^x	4,237 ^x
<code>+asin</code>	2.07 μ s	54.8 μ s	205 ms	27 ^x	99,130 ^x
<code>+acos</code>	2.10 μ s	51.2 μ s	5.17 ms	24 ^x	2,459 ^x
<code>+sqrt</code>	1.92 μ s	10.3 μ s	73.5 μ s	5 ^x	38 ^x
<code>+cbt</code>	2.44 μ s	467 μ s	111 ms	192 ^x	45,723 ^x
<code>+pow</code>	2.87 μ s	656 μ s	1.68 ms	229 ^x	585 ^x
<code>+pow-n</code>	2.62 μ s	217 μ s	219 μ s	83 ^x	83 ^x
<code>+log-2</code>	2.34 μ s	164 μ s	13.3 ms	70 ^x	5,668 ^x
<code>+log-10</code>	2.38 μ s	165 μ s	13.4 ms	69 ^x	5,638 ^x

Table 4: Elapsed times and relative slowdowns of unum jettied Chebyshev functions for 32-bit π ops positions2 operations with /1b/unum, interpreted Chebyshev functions, jettied Taylor series functions, and interpreted Taylor series functions. Slowdown is reported relative to the jettied Chebyshev functions.

Arm	Chebyshev (jet)	Chebyshev (int.)	Taylor (jet)	Taylor (int.)	C. slowdown	T. slowdown
+exp	16.5 μ s	550 μ s	25.3 μ s	19.4 ms	33 $\times$	1,176 $\times$
+log	17.9 μ s	739 μ s	39.9 μ s	39.1 ms	41 $\times$	2,184 $\times$
+sin	18.3 μ s	813 μ s	34.8 μ s	32.3 ms	44 $\times$	1,765 $\times$
+cos	18.3 μ s	817 μ s	35.7 μ s	32.5 ms	45 $\times$	1,775 $\times$
+tan	19.5 μ s	844 μ s	—	—	43 $\times$	—
+atan	18.3 μ s	832 μ s	850.3 μ s	63.7 ms	45 $\times$	3,480 $\times$
+asin	39.6 μ s	2.01 ms	—	—	50 $\times$	—
+acos	40.9 μ s	2.15 ms	—	—	52 $\times$	—
+sqt	17.1 μ s	209 μ s	17.3 μ s	341 μ s	12 $\times$	20 $\times$
+cbrt	32.0 μ s	1.76 ms	—	—	55 $\times$	—
+pow	31.2 μ s	1.40 ms	60.7 μ s	56.9 ms	45 $\times$	1,824 $\times$
+pow-n	2.81 μ s	698 μ s	—	—	248 $\times$	—
+log-2	18.1 μ s	743 μ s	—	—	41 $\times$	—
+log-10	18.1 μ s	756 μ s	—	—	42 $\times$	—
+fddp	259 ns	157 μ s	270 ns	205 μ s	606 $\times$	—

quire much higher intermediate precision to attain.²⁰ Tables 5 and 6 report the maximum observed ULP error of every transcendental arm against an independent mpmath-checked oracle.²¹

In Tables 5 and 6, the superscripts carry the following meanings:

- ^c marks a composed arm,²² whose error is inherited from its constituents' chained roundings rather than directly controlled.
- ^e marks an exhaustive sweep over the entire input space. (Unmarked cells describe sampled ranges.)
- ^f marks where an imprecision in a previous version of the Chebyshev fit led to massive error in the reduced argument, in which the range reduction quotient for `@rh` was not representable in half precision (Table 7). This was replaced by widening the intermediate result to `@rs`, producing a faithful rounding for `@rh` as well. As this bug results from a naïve application of the Chebyshev fit to half-precision range reduction, we report it as a cautionary aside.
- ^k marks dedicated kernels for `+tan` (replacing the composed `sin/cos` ratio used at `@rh/@rq`). A dedicated kernel was investigated but found to regress `@rh` (the 11-bit mantissa leaves insufficient headroom) and to need

²⁰See CR-LIBM (Daramy et al., 2003), RLIBM (Lim and Nagarakatte, 2021), and the CORE-MATH project (CORE-MATH Project, 2022) for further details.

²¹`libmath/tools/cheb_check.py` and `unum_cheb_check.py`. mpmath independently verifies the algorithm-of-record. Half precision IEEE 754 and all three posit widths are tested exhaustively over their entire input space; single, double, and quadruple precision (`@rs/@rd/@rq`) are sampled over a broad range with edge weighting near breakpoints and powers of two, since exhaustive testing is infeasible at those widths.

²²The arms `+pow`, `+cbrt`, and integer-exponent `+pow-n` beyond the cheap exact cases are built from `+exp/+log` or repeated multiplication rather than an independently minimax-fit kernel; `/lib/unum's +asin/+acos` are likewise composed, as `atan( $\frac{x}{\sqrt{1-x^2}}$ )` (and the corresponding identity for `+acos`) rather than a dedicated rational kernel—unlike `/lib/math's +asin/+acos`, which use one, and so are unmarked in Table 5.

- 456 a different technique entirely at `@rq` (the Chebyshev se-
 457 ries coefficients grow rather than shrink at high degree
 458 causing numerical instability).
- 459 • `^n` marks `/lib/unum's +cbt`, which returns NaR for nega-
 460 tive inputs and so is swept over positive inputs only.

Table 5: Maximum observed ULP error, `/lib/math` (IEEE 754), against an `mpmath`-checked oracle. `@rh` and marked cells are exhaustive over the full input space; all others are sampled.

Arm	@rh	@rs	@rd	@rq
+exp	1.00 ^e	0.86	0.99	0.81
+log	0.84 ^e	0.61	0.99	0.62
+log-2	1.57 ^e	0.90	1.21	0.60
+log-10	1.37 ^e	1.30	1.46	1.10
+sin	0.51 ^{e,f}	0.65	0.73	0.73
+cos	0.53 ^{e,f}	0.53	0.76	0.72
+tan	1.64 ^{e,f,c}	0.94 ^k	0.76 ^k	1.67 ^c
+atan	0.74 ^e	0.58	0.71	0.53
+atan2	1.80	1.16	1.36	1.35
+asin	1.01 ^e	0.85	0.84	0.77
+acos	1.00 ^e	0.98	0.89	0.86
+sqrt	0.50 ^e	0.50	0.50	0.50
+cbt	6.70 ^{c,e}	2.62 ^c	4.39 ^c	4.29 ^c
+pow	31.0 ^c	48.0 ^c	133.7 ^c	60.7 ^c

461 The original Chebyshev-based implementation of the `@rh`
 462 transcendental functions had a subtle but catastrophic flaw in
 463 its range reduction. The `@rh` Chebyshev fits were correct, but
 464 the range reduction was performed entirely in half precision,
 465 which is insufficient for large arguments. `@rh`'s trigonometric
 466 arms did quarter-turn range reduction ($q = \text{round}(|x| \cdot \frac{2}{\pi}) =$
 467 $|x| \cdot \frac{4}{\pi}$, $r = |x| - q \cdot (\frac{\pi}{2}) = |x| - q \cdot (\frac{\pi}{4})$) entirely in native `@rh`
 468 arithmetic, giving every reduction constant and every inter-
 469 mediate product only an 11-bit-mantissa half-precision value.
 470 Half precision guarantees exact integer representation only up
 471 to 2,048; once q exceeds that value ($|x|$ past ~ 500), q 's own

Table 6: Maximum observed ULP error, `/lib/unum` (2022-standard posits), against an `mpmath`-checked oracle. All three widths are tested exhaustively.

Arm	<code>@rpb</code>	<code>@rph</code>	<code>@rps</code>
<code>+exp</code>	0	0	0
<code>+log</code>	0	0	0
<code>+log-2</code>	0	0	0
<code>+log-10</code>	0	0	0
<code>+sin</code>	0	0	0
<code>+cos</code>	0	0	0
<code>+tan</code>	0	0	0
<code>+atan</code>	0	0	0
<code>+asin</code>	1 ^c	1 ^c	3 ^c
<code>+acos</code>	1 ^c	7 ^c	4 ^c
<code>+cbt</code>	1 ⁿ	2 ⁿ	2 ⁿ
<code>+pow</code>	3 ^c	4 ^c	4 ^c
<code>+pow-n</code>	2 ^c	2 ^c	2 ^c

472 rounding error was amplified by the $\frac{\pi}{2} = \frac{\tau}{4}$ leading term, in-
473 jecting a multi-radian error into the reduced remainder. Ta-
474 ble 7 reports the resulting damage, measured before the fix:
475 catastrophic, and well within `@rh`'s ordinary input range (max
476 magnitude $\sim 65,504$), not an edge case. The fix widens to `@rs`
477 (24-bit mantissa, exact q up to 2^{24} , vastly beyond the dynamic
478 range of `@rh`) via `+widen-hs` (exact) and `+narrow-sh` (correctly
479 rounded), primitives. These were verified exhaustively.

480 Every `/lib/unum` transcendental with a dedicated, non-
481 composed kernel is exactly correctly rounded (0 ULP) at all
482 three tested widths, a stronger guarantee than the IEEE 754
483 `/lib/math` achieves at any precision, where the best case is
484 faithful rounding around 0.5–1 ULP rather than 0 ULP. The com-
485 posed arms (`+pow`, `+cbt`, and `+pow-n` beyond small positive ex-
486 ponents) lose this property in both libraries alike, landing in
487 the 1–134 ULP range depending on precision and the number
488 of chained roundings the composition involves—an inherent
489 cost of composition rather than a library-specific weakness.

Table 7: @rh +sin/+cos/+tan, maximum observed ULP error before the range-reduction fix (Table 5 reports the corrected, current values).

Arm	@rh, before fix
+sin	437,291
+cos	1,818,711
+tan	2.16×10^{10}

6.2 The price of determinism

The third field of benchmarking measures the overhead of the determinism guarantee relative to a conventional, nondeterministic, hardware BLAS. We report SoftBLAS against Apple’s Accelerate framework—a stock BLAS on macOS which dispatches to Apple Silicon’s AMX matrix coprocessor—for sgemm and dgemm (single-threaded to remove Accelerate’s automatic multithreading as a confound). We expect software-defined SoftBLAS to be slower; here we quantify how that cost scales. The results are summarized in Table 8 and Figure 1.

*At $n \leq 20$ Accelerate’s own run-to-run standard deviation is comparable to its mean, so the reported slowdown there is noise-dominated; the trend is reliable and monotonic above $n = 30$ (SoftBLAS’s timings are stable throughout since a fixed triple loop has no comparable startup-dominated regime).

Determinism does not levy a fixed tax, but its cost grows with problem size, from single digits at $n = 10$ to nearly four orders of magnitude by $n = 1,000$ (sgemm) or over three orders of magnitude (dgemm). SoftBLAS’s triple-nested loop over SoftFloat calls is a clean $O(n^3)$ throughout the range tested (doubling n consistently multiplies time by $\sim 8\times$), with no cache blocking or vectorization—each scalar multiply-add pays a full software floating-point call.²³ Accelerate is sub-cubic through the moderate range (cache blocking and SIMD/AMX dispatch absorb work that would otherwise scale as n^3) before trending

²³Other matrix multiplication algorithms, such as Strassen’s algorithm, can reduce the asymptotic complexity, but they are not used in this context.

back toward cubic by $n = 1,000$. In throughput terms, Soft-BLAS delivers roughly 0.12 GFLOP/s at $n = 1,000$ regardless of precision—consistent with SoftFloat call overhead, not memory bandwidth, being the bottleneck—against Accelerate’s approximately 1.477 GFLOP/s (sgemm) and 401 GFLOP/s (dgemm), the latter gap itself illustrating that Accelerate’s single-precision path better exploits the AMX unit’s native format.

This sharply demonstrates a quantitative trade-off between determinism and performance introduced in Section 3.1: Soft-BLAS is not merely “slower,” but slower by a margin that widens with the problem sizes—large dense linear algebra—where BLAS exists to be fast in the first place. A deterministic numerical stack should budget accordingly. A software BLAS that never touches a hardware floating-point instruction cannot reach a coprocessor-backed BLAS’s throughput, only close the distance to it, which argues in favor of new paradigms for deterministic numerical computing that can leverage hardware acceleration without sacrificing reproducibility.

7 Future Work

The complex-valued SoftBLAS routines are in progress and will complete the Level 1–3 coverage. Quire operations in `/lib/unum` are implemented within the generic posit core but are not yet surfaced as callable top-level arms; exposing them is a prerequisite both for user access and for routing them through the verification harness. The unum specification also defines `valids`, which are arithmetic intervals which have not yet seen widespread adoption.

Saloon will grow in methods adjacent to scientific computing: conjugate gradient minimization and other optimization algorithms; random number generators; and the like. The fixed-point library is relatively underdeveloped compared to the other arithmetics, and will be expanded to support more general Q-format widths and operations. More components of the various libraries need to be upgraded to a state-of-the-art Chebyshev basis and benchmarked for performance.

Finally, Maroon, the machine learning library, is in its in-

Table 8: sgemm/dgemm mean per-call time, SoftBLAS vs. Apple Accelerate, single-threaded, Apple M4 Max. n is the (square) matrix dimension.

n	SoftBLAS sgemm	Accelerate sgemm	slowdown	SoftBLAS dgemm	Accelerate dgemm	slowdown
10	21.9 μ s	7.3 μ s	3.0 $\times$	20.8 μ s	4.7 μ s	4.4 $\times$
20	160 μ s	9.0 μ s	18 $\times$	151 μ s	1.9 μ s	80.0 $\times$
30	521 μ s	0.79 μ s	662 $\times$	496 μ s	1.4 μ s	354 $\times$
40	1.20 ms	1.4 μ s	873 $\times$	1.16 ms	2.3 μ s	496 $\times$
50	2.27 ms	1.9 μ s	1,202 $\times$	2.22 ms	4.1 μ s	545.7 $\times$
75	7.44 ms	3.2 μ s	2,322 $\times$	7.25 ms	7.1 μ s	1,020.4 $\times$
100	17.3 ms	5.8 μ s	2,970 $\times$	17.8 ms	13.9 μ s	1,282.3 $\times$
200	133 ms	22.5 μ s	5,921 $\times$	135 ms	57.9 μ s	2,342 $\times$
500	2.02 s	203 μ s	9,947 $\times$	2.02 s	675 μ s	3,000 $\times$
1000	16.05 s	1.35 ms	11,853 $\times$	16.25 s	4.99 ms	3,257 $\times$

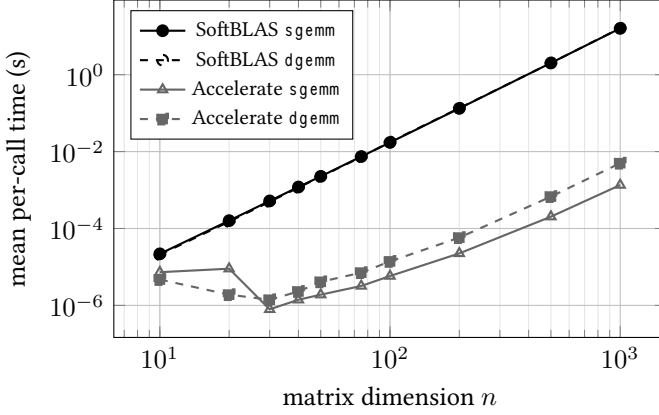


Figure 1: sgemm/dgemm mean per-call time vs. matrix dimension n , log-log, SoftBLAS vs. Apple Accelerate (data from Table 8). SoftBLAS’s clean $O(n^3)$ scaling appears as a straight line of slope 3 throughout (sgemm and dgemm nearly coincide); Accelerate’s sub-cubic scaling at moderate n visibly bends below that trend before steepening again near $n = 1,000$ —the widening vertical gap between the two families is the determinism tax growing with problem size. The Accelerate points at $n \leq 30$ are noise-dominated (Table 8, note *), hence the irregular dip rather than a smooth curve at small n .

551 fancy and will be developed to support a range of machine
 552 learning algorithms and techniques. Part of this field of con-
 553 sideration encompasses the utilization of GPU programming for
 554 LLM-based computing within or adjacent to Urbit. GPU numeri-
 555 cal nondeterminism and host portability presents a challenge
 556 of the same order of magnitude as the baseline reproducibil-
 557 ity we have tackled over the past several years in this project.
 558 One can envision a ReproCL as a deterministic counterpart to
 559 OpenCL, but it is unclear whether sufficient warrant exists to
 560 justify the effort as of the mid-2020s. Heterogeneous com-
 561 pute would be benefitted by the adoption of alternatives to IEEE
 562 754 with better guarantees of portability: not just unums but
 563 bfloat16 and other newer types may answer the need.

8 Conclusion

Urbit as a deterministic state machine cannot treat numerical determinism or bitwise reproducibility as merely a tolerance. The architectural stack we have described here constitutes a partial answer to the challenge underlying `~lagrev-nocfep` (2024): how can a numerical computation built on a particular stack be reliably reproducible? We maintain this determinism by painstaking construction. The computational cost of determinism is currently (and regrettably) non-negligible, while one can hope for future developments, protocols, and architectures that make it more natural and tenable. ☹

References

- Agullo, Emmanuel et al. (2009). “Numerical Linear Algebra on Emerging Architectures: The PLASMA and MAGMA Projects.” In: *Journal of Physics: Conference Series* 180, p. 012037. DOI: 10.1088/1742-6596/180/1/012037.
- Charney, J., R. Fjortoft, and J. von Neumann (1950). “Numerical Integration of the Barotropic Vorticity Equation.” In: *Tellus* 2.4, pp. 237–254. DOI: 10.1111/j.2153-3490.1950.tb00336.x. URL: <https://doi.org/10.1111/j.2153-3490.1950.tb00336.x>.
- Cody, William J. and William M. Waite (1980). *Software Manual for the Elementary Functions*. Prentice-Hall.
- CORE-MATH Project (2022) “CORE-MATH: Correctly Rounded Mathematical Functions”. URL: <https://core-math.gitlabpages.inria.fr/>.
- Daramy, Catherine et al. (2003). “CR-LIBM: A Correctly Rounded Elementary Function Library.” In: *Proc. SPIE 5205, Advanced Signal Processing Algorithms, Architectures, and Implementations XIII*. DOI: 10.1117/12.505591.
- Demmel, James and Hong Diep Nguyen (2013). “Fast Reproducible Floating-Point Summation.” In: *2013 IEEE 21st Symposium on Computer Arithmetic (ARITH)*. ReproBLAS project, pp. 163–172. DOI: 10.1109/ARITH.2013.9.

- 598 Fousse, Laurent et al. (2007). “MPFR: A Multiple-Precision
599 Binary Floating-Point Library with Correct Rounding.” In:
600 *ACM Transactions on Mathematical Software* 33.2,
601 13:1–13:15. DOI: 10.1145/1236463.1236468.
- 602 Free Software Foundation (2024). *The GNU C Library Reference*
603 *Manual*. Section 19.7, Known Maximum Errors in Math
604 Functions. URL: [https://www.gnu.org/software/libc/
605 manual/html_node/Errors-in-Math-Functions.html](https://www.gnu.org/software/libc/manual/html_node/Errors-in-Math-Functions.html).
- 606 Hauser, John R. (2018) “Berkeley SoftFloat, Release 3e”.
607 *IEEE Standard for Floating-Point Arithmetic* (2019). IEEE Std
608 754-2019. IEEE. DOI: 10.1109/IEEESTD.2019.8766229.
- 609 Intel Corporation (2023) “Conditional Numerical
610 Reproducibility (CNR) in Intel oneAPI Math Kernel
611 Library”.
- 612 ~lagrev-nocfep, N. E. Davis (2024). “The Desert of the Reals:
613 Floating-Point Arithmetic on Deterministic Systems.” In:
614 *Urbis Systems Technical Journal* 1.1, pp. 93–131.
- 615 — (2026) “SoftUnum: Software-Defined Universal Number
616 Library for Deterministic Jetting”. URL:
617 <https://github.com/sigilante/SoftUnum>.
- 618 Lawson, C. L. et al. (1979). “Basic Linear Algebra Subprograms
619 for Fortran Usage.” In: *ACM Transactions on Mathematical*
620 *Software* 5.3, pp. 308–323. DOI: 10.1145/355841.355847.
- 621 Leong, S. H. (Cerlane) (2018) “SoftPosit: A Software
622 Implementation of Posit Arithmetic”.
- 623 Lim, Jay P. and Santosh Nagarakatte (2021). “High
624 Performance Correctly Rounded Math Libraries for 32-bit
625 Floating Point Representations.” In: *Proc. 42nd ACM*
626 *SIGPLAN Conf. on Programming Language Design and*
627 *Implementation (PLDI)*, pp. 359–374. DOI:
628 10.1145/3453483.3454049.
- 629 Muller, Jean-Michel et al. (2018). *Handbook of Floating-Point*
630 *Arithmetic, 2nd edition*. Birkhäuser Boston, p. 632.
- 631 Omtzigt, E. Theodore L. et al. (2020). “Universal Numbers
632 Library: Design and Implementation of a High-
633 Performance Reproducible Number Systems Library.” In:
634 *arXiv preprint arXiv:2012.11011*. URL:
635 <https://arxiv.org/abs/2012.11011>.

- 636 Posit Working Group (2022). *Standard for Posit Arithmetic*
637 (2022). Posit Working Group. URL:
638 https://posithub.org/docs/posit_standard-2.pdf.
639 Rump, Siegfried M., Takeshi Ogita, and Shin'ichi Oishi (2008).
640 “Accurate Floating-Point Summation Part I: Faithful
641 Rounding.” In: *SIAM Journal on Scientific Computing* 31.1,
642 pp. 189–224. DOI: 10.1137/050645671. URL:
643 <https://doi.org/10.1137/050645671>.
644 ~sorreg-namtyv, Curtis Yarvin et al. (2016). *Urbit: A*
645 *Solid-State Interpreter*. Whitepaper. Tlon Corporation. URL:
646 <https://media.urbit.org/whitepaper.pdf> (visited on
647 ~2024.1.25).
648 Urbit Foundation (2024) “SoftBLAS: Software-Defined BLAS
649 for Deterministic Jetting”. URL:
650 <https://github.com/urbit/SoftBLAS>.